1. Основные состояния запроса

При выполнении HTTP-запроса удобно использовать состояния, чтобы показывать пользователю загрузку, результат или ошибку.

Состояние	Значение
`idle`	Запрос ещё не отправлялся
`loading`	Запрос выполняется
`success`	Запрос завершился успешно, получены данные
`error`	Запрос завершился ошибкой
`aborted`	Запрос отменён вручную

`aborted` — отдельное состояние, которое удобно отличать от обычной ошибки. Например, пользователь ввёл новый поисковый запрос, поэтому старый запрос больше не нужен.

2. AbortController и отмена запроса

`AbortController` — объект JavaScript, который позволяет отменять асинхронные операции, в том числе запросы через `fetch()`.

Основные элементы:

- `new AbortController()` — создаёт контроллер.
- `controller.signal` — сигнал, который передаётся в запрос.
- `controller.abort()` — отменяет запрос.

Пример:

```
const controller = new AbortController();

fetch('/api/users', {
    signal: controller.signal
});

controller.abort();
```

Если запрос отменён, `fetch()` обычно завершается с ошибкой `AbortError`. Её нужно отличать от сетевой ошибки, чтобы не показывать пользователю лишние сообщения.

Важно: `fetch()` не считает HTTP-статусы `404` и `500` исключениями. Поэтому необходимо самостоятельно проверять `response.ok`.

```
if (!response.ok) {
    throw new Error(`HTTP ${response.status}`);
}
```

3. Отмена запроса по тайм-ауту

Иногда сервер отвечает слишком долго. Для ограничения времени ожидания можно использовать `AbortSignal.timeout(ms)`.

```
async function fetchWithTimeout(url, ms = 5000) {
    const response = await fetch(url, {
        signal: AbortSignal.timeout(ms)
    });

    if (!response.ok) {
        throw new Error(`HTTP ${response.status}`);
    }

    return response.json();
}
```

В этом примере запрос отменяется по истечении заданного времени.

- `5000` — время ожидания в миллисекундах, то есть 5 секунд.
- `TimeoutError` — ошибка, связанная с истечением времени ожидания.

4. Объединение сигналов отмены

`AbortSignal.any([...])` объединяет несколько сигналов в один. Запрос отменяется, когда срабатывает любой из них.

Например, можно объединить:

- отмену по действию пользователя;
- отмену по тайм-ауту.

```
const controller = new AbortController();
const timeoutSignal = AbortSignal.timeout(5000);

const signal = AbortSignal.any([
```

```
    controller.signal,
    timeoutSignal
]);

fetch('/api/data', { signal });
```

Это позволяет использовать несколько причин отмены для одного запроса.

5. Гонки HTTP-запросов

Гонка запросов — ситуация, когда несколько запросов выполняются одновременно и ответы приходят не в том порядке, в котором запросы были отправлены.

Например, пользователь быстро вводит в поиске `a`, `ab`, `abc`. Ответ на первый запрос может прийти последним и перезаписать актуальные результаты.

Плохой вариант

```
async function searchBad(query) {
    const response = await fetch(
        `/api/search?q=${encodeURIComponent(query)}`
    );

    const data = await response.json();
    renderResults(data);
}
```

Здесь каждый ответ отображается независимо от того, актуален ли поисковый запрос. Поэтому на экране могут оказаться устаревшие результаты.

Решение № 1. Отмена предыдущего запроса

```
let activeController = null;

async function searchWithAbort(query) {
    if (activeController) {
        activeController.abort();
    }

    activeController = new AbortController();

    try {
        const response = await fetch(
            `/api/search?q=${encodeURIComponent(query)}`,
```

```
            { signal: activeController.signal }
        );

        if (!response.ok) {
            throw new Error(`HTTP ${response.status}`);
        }

        const data = await response.json();

        if (activeController.signal.aborted) return;

        renderResults(data);
    } catch (error) {
        if (error.name === 'AbortError') return;
        renderError(error);
    }
}
```

При новом запросе предыдущий отменяется. Это помогает экономить ресурсы и не обрабатывать ненужные ответы.

Решение № 2. Счётчик запросов

Можно присваивать каждому запросу свой номер. Если за время выполнения появился более новый запрос, старый результат игнорируется.

```
let requestId = 0;

async function searchWithSeq(query) {
    const myId = ++requestId;

    try {
        const response = await fetch(
            `/api/search?q=${encodeURIComponent(query)}`
        );

        if (!response.ok) {
            throw new Error(`HTTP ${response.status}`);
        }

        const data = await response.json();

        if (myId !== requestId) return;

        renderResults(data);
```

```
    } catch (error) {
        if (myId !== requestId) return;
        renderError(error);
    }
}
```

Счётчик гарантирует, что обработан будет только результат последнего запроса. При этом сам старый запрос не отменяется и продолжает использовать ресурсы.

Решение № 3. Отмена и счётчик одновременно

Это более надёжный вариант: отмена уменьшает ненужную работу, а счётчик защищает от обработки устаревших результатов.

В исходном коде это реализовано в функции `search()`:

1. Отменяется предыдущий запрос.
2. Создаётся новый `AbortController`.
3. Увеличивается номер запроса.
4. Включается состояние загрузки.
5. Отправляется запрос и разбирается JSON.
6. Проверяется актуальность результата.
7. Вызывается `onSuccess()` или `onError()`.
8. В `finally` снимается загрузка только для актуального запроса.

6. React: хук useFetch

`useFetch` — пользовательский React-хук, который выполняет HTTP-запрос и хранит его состояние.

В нём используются три инструмента:

- `useReducer` — управляет состоянием запроса.
- `useRef` — хранит `AbortController` между повторными рендерами.
- `useEffect` — запускает запрос при изменении URL и выполняет очистку.

Reducer

Reducer определяет, как меняется состояние при каждом действии:

Действие	Результат
`start`	`loading`, данные и ошибка очищаются

Действие	Результат
`success`	`success`, сохраняются полученные данные
`error`	`error`, сохраняется ошибка

Начальное состояние:

```
{
    status: 'idle',
    data: null,
    error: null
}
```

Как работает useFetch

1. При появлении URL создаётся контроллер.
2. Состояние меняется на `loading`.
3. Выполняется запрос `fetch()`.
4. Если ответ успешный, данные преобразуются из JSON.
5. Если возникает ошибка, состояние меняется на `error`.
6. При изменении URL или размонтировании компонента предыдущий запрос отменяется.

Важная часть:

```
return () => controller.abort();
```

Это функция очистки `useEffect`. Она помогает не выполнять ненужные запросы после смены URL или размонтирования компонента.

Компонент `UserProfile` использует этот хук и отображает:

- индикатор загрузки при `idle` или `loading`;
- сообщение об ошибке при `error`;
- имя и электронную почту при успешном получении данных.

7. Debounce — задержка выполнения

`debounce` откладывает выполнение функции до тех пор, пока пользователь не перестанет вызывать её в течение заданного времени.

Например, при задержке 300 мс поиск запускается не после каждого введённого символа, а после небольшой паузы.

```
function debounce(fn, delay) {
    let timerId = null;

    return function (...args) {
        clearTimeout(timerId);
        timerId = setTimeout(
            () => fn.apply(this, args),
            delay
        );
    };
}
```

В примере с поиском используются два механизма:

- `debounce` уменьшает количество запросов;
- `AbortController` отменяет предыдущий запрос, если пользователь продолжил ввод.

Эти механизмы дополняют друг друга, но не заменяют друг друга.

8. Защита от двойного нажатия на кнопку

При двойном нажатии на кнопку «Отправить» можно случайно отправить два одинаковых запроса.

В коде используются:

- `isSubmitting` — флаг, запрещающий повторную отправку;
- `button.disabled = true` — блокировка кнопки;
- `submitController` — контроллер отмены запроса;
- `finally` — восстановление состояния кнопки после завершения операции.

Алгоритм:

1. Проверить, выполняется ли уже отправка.
2. Если да — не запускать новый запрос.
3. Заблокировать кнопку и показать текст «Отправка…».
4. Выполнить запрос.
5. Показать успех или ошибку.
6. В `finally` разблокировать кнопку и вернуть исходный текст.

Это помогает избежать случайных повторных действий со стороны пользователя.

9. Функция safeFetch

`safeFetch` — универсальная обёртка над `fetch()`, которая упрощает обработку запросов.

Она выполняет следующие задачи:

- задаёт тайм-аут;
- поддерживает пользовательскую отмену;
- проверяет HTTP-статус;
- пытается извлечь сообщение из тела ошибки;
- разбирает JSON или обычный текст в зависимости от `Content-Type`;
- преобразует сетевые ошибки в понятные сообщения.

Если запрос отменён пользователем, функция возвращает объект:

```
{ aborted: true }
```

При этом тайм-аут должен отличаться от пользовательской отмены.

Замечание к исходному коду: проверка `AbortError` внутри `safeFetch` не всегда достаточна для определения тайм-аута. В современных реализациях отмена через `AbortSignal.timeout()` обычно приводит к `TimeoutError`. Поэтому обработку тайм-аута и пользовательской отмены лучше делать с учётом фактической причины отмены.

10. Паттерн «последний запрос побеждает»

`createLatestOnly()` создаёт функцию, которая возвращает результат только последнего вызова.

Она использует два механизма:

- `AbortController` — для отмены предыдущего запроса;
- счётчик `id` — для проверки актуальности результата.

Пример использования:

```
const latest = createLatestOnly();

async function loadUser(id) {
    return latest(async (signal) => {
        const response = await fetch(
```

```
            `/api/users/${id}`,
            { signal }
        );

        if (!response.ok) {
            throw new Error(`HTTP ${response.status}`);
        }

        return response.json();
    });
}

loadUser(1);
loadUser(2);
loadUser(3);
```

Если вызовы происходят быстро, актуальным считается последний — `loadUser(3)`. Результаты предыдущих запросов игнорируются, если к моменту их обработки уже появился новый запрос.

Кратко для запоминания

Технология	Для чего нужна
`fetch()`	Отправляет HTTP-запрос
`AbortController`	Позволяет отменить запрос
`AbortSignal.timeout()`	Отменяет запрос по тайм-ауту
`AbortSignal.any()`	Объединяет несколько сигналов отмены
`response.ok`	Проверяет, успешен ли HTTP-ответ
`useEffect`	Запускает запрос и выполняет очистку в React
`useReducer`	Управляет состояниями запроса
`useRef`	Хранит контроллер между рендерами
`debounce`	Сокращает число вызовов при частых событиях
Счётчик запросов	Не позволяет устаревшему ответу обновить интерфейс
`finally`	Выполняет завершающие действия независимо от результата

Главный вывод: `AbortController` отвечает за отмену запросов, счётчик защищает от устаревших результатов, `debounce` уменьшает количество запросов, а состояния `idle`,

`loading`, `success` и `error` помогают правильно отображать процесс работы в интерфейсе.